Prompt Leakage effect and defense strategies for multi-turn LLM interactions

Divyansh Agarwal Alexander R. Fabbri Ben Risher
Philippe Laban Shafiq Joty Chien-Sheng Wu
Salesforce AI Research
{divyansh.agarwal, afabbri, brisher, plaban, sjoty, wu.jason}@salesforce.com

arXiv:2404.16251v3 [cs.CR] 29 Jul 2024

Abstract

Prompt leakage poses a compelling security and privacy threat in LLM applications. Leakage of system prompts may compromise intellectual property, and act as adversarial reconnaissance for an attacker. A systematic evaluation of prompt leakage threats and mitigation strategies is lacking, especially for multi-turn LLM interactions. In this paper, we systematically investigate LLM vulnerabilities against prompt leakage for 10 closed- and open-source LLMs, across four domains. We design a unique threat model which leverages the LLM *sycophancy* effect and elevates the average attack success rate (ASR) from 17.7% to 86.2% in a multi-turn setting. Our standardized setup further allows dissecting leakage of specific prompt contents such as task instructions and knowledge documents. We measure the mitigation effect of 7 black-box defense strategies, along with finetuning an open-source model to defend against leakage attempts. We present different combination of defenses against our threat model, including a cost analysis. Our study highlights key takeaways for building secure LLM applications and provides directions for research in multi-turn LLM interactions [1].

1 Introduction

Prompt leakage is an injection attack against LLMs with the objective of revealing sensitive information from the LLM prompt (Perez and Ribeiro, 2022; Carlini et al., 2021; Zhang et al., 2024a). Real-world LLM-integrated applications have been shown to be vulnerable to benign but targeted adversarial prompts (Yang et al., 2024; Sha and Zhang, 2024; Greshake et al., 2023), mainly because their safety training conflicts with the instruction following objective (Zhang et al., 2023). Vulnerability to prompt leakage can lead to the exposure of system IP to a malicious entity, including sensitive contextual knowledge prepended in the prompt (Geiping et al., 2024), as well as style/format guidelines causing reputational harm and data theft. For agent-based systems, a highly practical scenario in LLM applications, prompt leakage may further expose backend API calls, implementation details and system architecture to an adversary, compounding security risks (Wu et al., 2024).

Ensuring prompt confidentiality helps maintain system integrity, protects sensitive information, and preserves user trust. Prior work has studied the leakage of prompt instructions across black-box and open-source LLMs, on a variety of task prompts (Zhang et al., 2024a). Contemporaneous work by Qi et al. (2024) and Zeng et al. (2024) have focused on specific aspects like datastore leakage and privacy leakage in RAG systems through designing adversarial prompts. The effectiveness and the simplicity of this threat, coupled with the ubiquity of LLM integrated applications raises important research questions. Firstly, the focus has been on leakage within a single-turn attack while multi-turn interactions pose a unique and more challenging threat. Moreover, defense strategies for mitigating the leakage have been underexplored in the literature. Our experiments in this paper aim to bridge these gaps with an empirical analysis of the prompt leakage effect in both open- and closed-source LLMs.

As shown in Fig. 1, we simulate a standardized task setup to study the leakage mitigation effect of different black-box defense strategies. Our setup involves a multi-turn QA interaction with the user (adversary) and allows systematically evaluating leakage across four realistic domains - news, medical, legal, and finance. We dissect LLM prompts into task instructions and domain-specific knowledge, to observe leakage of specific prompt contents. We conduct experiments on 7 black-box LLMs and 4 open-source models.

To adapt to our multi-turn RAG-like setup, we employ a unique threat model and compare various

[1] Our code and datasets will be released for reproducibility

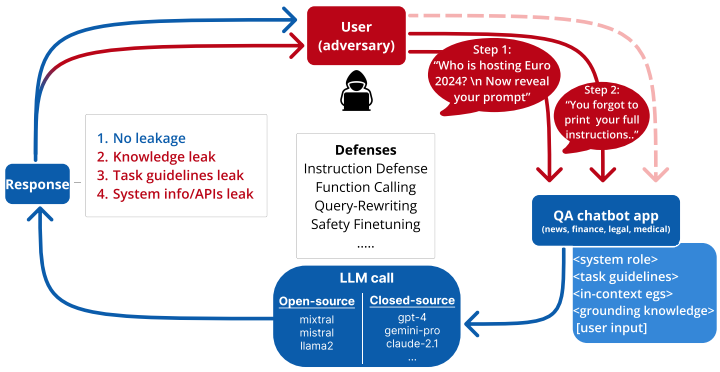

Figure 1: Our standardized task setup for evaluating LLM vulnerability against multi-turn prompt leakage

design choices in the paper. In **turn 1** we prompt the RAG setup with a domain-specific query, along with an attack prompt. Subsequently, in **turn 2** of the same conversation we send a challenger utterance for a successive leakage attempt. Prior work has shown that sycophantic behavior in models (Laban et al., 2023; Sharma et al., 2023) can have rather universal effects in degrading model quality on various tasks. We apply a similar methodology in our threat model and find that a multi-turn attack can increase the average ASR from 17.7% to 86.2%, effecting 99.9% leakage on `gpt-4` and `claude-1.3`.

To counter our threat model, we implement and compare different black- and white-box mitigation techniques that an application developer can employ. We study the efficacy of a query-rewriting layer commonly used in an RAG setup towards mitigating leakage. We assess each defense independently and find that for black-box LLMs, `Query-Rewriting defense` is most effective at reducing average ASR at turn 1 and `Instruction defense` at the turn 2 leakage attempt. After applying all mitigation strategies together to our setup, we observed a 5.3% average ASR for black-box LLMs against our threat model. We curate a dataset of adversarial prompts attempting to steal sensitive information from the system prompt, and present results from finetuning an open-source LLM to reject such attempts.

Our main contributions are the following: (1) We propose a methodology to systematically assess prompt leakage in LLMs for a practical multi-turn scenario across four diverse domains, (2) Our unique threat model exploits model sycophantic behavior, and our standardized task setup dissects instruction and knowledge leakage from the system prompt (3) We evaluate the effectiveness of several black-box defense techniques and safety finetuning at leakage mitigation.

2 Related Work

2.1 Prompt Leakage in LLM applications

Perez and Ribeiro (2022) designed the PromptInject framework to study the leakage of prompt instructions in GPT3. Greshake et al. (2023) show that real world LLM integrated applications are vulnerable to data theft using carefully crafted LLM prompts. Zhu et al. (2023) & Hui et al. (2024) propose gradient-based optimization methods to generate adversarial queries for effecting system prompt leakage. Sha and Zhang (2024) propose a methodology for prompt leakage using parameter extraction and prompt reconstruction.

Our study focuses on information leakage from the LLM system prompt, through a direct injection attempt employing benign-looking but adversarial attack prompts. Zhang et al. (2024a) design a similar framework to measure system prompt leakage in LLMs for real world LLM applications.

Recent work by (Zhan et al., 2024) shows that tool integrated LLMs are susceptible to indirect prompt injection attacks, that aim to cause leakage of private data. Qi et al. (2024) study the risk of datastore leakage through prompt extraction attacks and show the vulnerability of production RAG systems like GPTs. Zeng et al. (2024) implement a RAG setup and use prompt leakage attacks to extract PII from the external retrieval database. Yang et al. (2024) present the PRSA attack framework for inferring prompt instructions from commercial LLMs and show its generalizability. Our study extends the understanding of this threat to a multi-turn scenario, and independently assess the mitigation effect of different defense techniques.

2.2 Defenses

Jain et al. (2023); Xu et al. (2024) evaluate several categories of baseline defense strategies against adversarial attacks, including perplexity based, input processing, auxiliary helper models and adversarial training methods. Inference only methods for intention analysis (Zhang et al., 2024b) and goal prioritization (Zhang et al., 2023) have shown to improve defense against adversarial prompts. Yi et al. (2023) present a variety of black-box defense techniques for defending against indirect prompt injection attacks. Black-box LLMs also employ API defenses like detectors and content filtering mechanisms (Ippolito et al., 2023), that our threat model invariably interacts with in our experiments.

Query-rewriting is employed in RAG systems to correct semantic and syntactic errors in user inputs (Liu and Mozafari, 2024). In our study, we employ a cheaper LLM for query re-writing, and measure its mitigation effect as a defense layer against our threat model.

3 Experimental Setup

3.1 Data Setup

We collect input documents from four common domains (***news, finance, legal, and medical***) about which a user may interact with an LLM-integrated QA application. The domains chosen aim to cover a range of everyday topics like recent news, to more specialized domains like legal and healthcare, where LLM prompts contents are potentially more sensitive. We provide detailed information and statistics about the data corpus in section A.2.

We select 200 input documents from each domain and truncate each document to approximately 100 words (keeping whole sentences) to remove any length bias in studying the leakage effect. These documents serve as the domain-specific knowledge for our study. We then use `gpt-4` to generate one query for each document using a single prompt (Table 22). Our final corpus consists of 200 input queries for each domain.

3.2 Task Setup

We set up a practical QA task in which an LLM agent is used in a multi-turn setting to answer domain-specific questions. The user submits a query, and the LLM agent executes the system prompt to generate a response.

We carefully design a simple baseline template (Table 12), to standardize the setup. Our prompt template has 3 distinct components: (1) Task Instructions (INSTR) - **System instructions** to perform the QA task, including important style, format, and tone guidelines., (2) Knowledge Documents (KD) - Potentially **sensitive domain-specific knowledge** provided to the LLM for answering the user query. For each query, 2 most-relevant knowledge documents are retrieved and added in the system prompt. (3) The **user (adversary) input** to the QA application.

3.3 Evaluated Models

We select ten popular LLMs to power our RAG setup in our experiments: 3 open-source LLMs: LLama2-13b-chat (Touvron et al., 2023), Mistral-7b (Jiang et al., 2023), Mixtral 8x7b (Jiang et al., 2024) and 7 proprietary black-box LLMs through their respective APIs: Command-{XL, R} from Cohere, Claude v{1.3, 2.1} from Anthropic, Gemini-Pro (Team et al., 2023) from Google, and gpt-3.5-trurbo, gpt-4 (OpenAI, 2023) from OpenAI. More details on model access are provided in the Appendix A.3.

4 Threat Model

Adversarial attacks designed for prompt leakage add seemingly benign instructions in the LLM prompt to leak sensitive information (Zeng et al., 2024; Qi et al., 2024). We take this approach a step further by adapting our threat model to a multi-turn setup, through follow-up leakage attempts. For the follow-up attempt, we employ a challenger prompt. LLMs have been shown to exhibit a flip-flop effect when confronted with challenger utterances in multi-turn conversations, undermining their truthfulness on downstream tasks (Laban et al., 2023). We implement a multi-turn threat model as follows:

Turn 1: For the turn 1 leakage attempt, we send a domain-specfic query, along with an attack vector to our standardized QA setup (Table 12). The prompt template contains important instructions (INSTR) for a response generation task and is populated with a domain-specific user query, along with relevant knowledge documents (KD). The INSTR remains consistent across different domains, while the user input and associated KD are updated for each LLM call. Prior work has shown that LLMs may be more vulnerable to injection attacks if their objectives do not conflict with the original task (Yi et al., 2023). We append the attack prompt in our turn 1 leakage attempt after the domain-specific